

Final Project

Ethan Scoggins, Keenrruy Murillo
Computer Science & Engineering
Speed School of Engineering
University of Louisville, USA
esscog01@louisville.edu, ksmuri01@louisville.edu

Introduction

This project is a culmination of all our previous work in the course for problem solving. By leveraging our experiences in TSP in particular, we are expected to research a different problem, and implement an algorithm that either solves it or finds the closest solution. Previously, by using the genetic algorithm to cycle through a random assortment of individuals that represent possible solutions, ranking them by fitness, and building off the best individuals, we were able to advance to an optimal solution. Then when implementing the hybrid Wisdom of crowds approach and utilizing the different experts from different crowds, we were able to further push the solutions to use parts of these experts into both exploring other possibilities and maintaining a high fitness. With these concepts, we are able to create a new program to solve an NP-complete problem of our choice and visualize it within a GUI.

Regarding combinatorial optimization, the N-Queens completion puzzle represents a classic opportunity to solve an NP-complete problem. The challenge revolves around placing a number of Chess queens N on a chessboard of size $N \times N$ in such a manner that no queen poses an immediate threat to another. This problem sees exponential growth in computational complexity given the nature of the challenge wherein the chessboard grows in size in accordance with any increases to the number of queens. It is for this reason that traditional algorithms such as brute force search are not ideal for deciphering such a problem. As such, in this project, the primary goal is to develop an algorithm that has the ability to solve the N-Queens problem for an optimal or near-optimal solution within a reasonable time frame. To achieve this end, an innovative approach utilizing a hybrid genetic algorithm paired with wisdom of artificial crowds was developed and implemented via Python. This approach allows for the computational benefits of the genetic algorithm to be further expanded upon by utilizing collective intelligence to better discover the most optimal answer possible. Furthermore, for increased user experience, a

graphical user interface was to be created through which parameters specific to the problem could be defined and the final arrangement of queens could be visually displayed.

Approach

The program is divided into two files, GAWOCMain.py, and GAWOCUtil.py. The former houses the initialization parameters for the “geneticalgorithm” function, and the code for the GUI visualization. The latter houses the WOC implementation, Genetic Algorithm implementation, corresponding functions for populations, crossovers, mutations, and variables to keep track of the experts.

In greater depth, we start in the main file and call the genetic algorithm function in the util file, telling it to run until we find a solution. The first WOC loop is run, variables to keep track of the best performing individuals overall and by generation are initialized here. We also initialize a population variable using an initializepop function that creates a list of lists full of integers, the size of N, randomly.

The nested generations loop begins afterwards. We find the fitness scores of this population and place them into a list called fitscores. The fitnesses are found individually for each chromosome with the evalfitness function, which takes an individual list and the number of queens; By setting a counter equal to the number of queens, finding the row column values for a singular queen, then looping through all other queens, we can check if the current queen being checked is a validly positioned queen. If the row or column is found to be the same then it is marked as invalid. To check diagonally, an if statement checks if the absolute value of the difference in the two queens columns and rows are equal, and if they are, then the queens being checked share a diagonal square, which then marks the queen as invalid. When a queen is marked invalid, 1 is subtracted from the counter and the loop is broken, then checking for a new set of queens. This counter is divided by the total N queens passed, giving a fitness value between 0 and 1, where this ratio is the percentage of validly positioned queens in the chromosome/individual.

With the fitness scores found, we preserve the best individual from those fitness scores, and if this value for the current generation is better than our overall best individual, it is replaced with this newly found individual. Regardless of this, the best individuals and their fitnesses by generation are saved into a list for future reinitialization. Finally, the population is modified using the newgen function. The newgen function creates a fitness list of the current population being passed, and uses it to randomly pick two parents from the population. The pick one function although choosing randomly, will choose a parent with greater fitness more often than those with lower fitness, still giving every individual a chance to be picked. From there, crossover and mutation functions are ran utilizing the selected parents and the child is appended to the newPopulation variable. This new population is then returned and replaces the population

within the generations loop. This being that last line within the nested generations loop, the loop continues until the criteria of fitness 1 is met or we have completed all generations.

With the generations nested loop complete, we continue running the outer WOC loop. An if statement within the loop, checking that if this is not the first woc loop, then reinitialize the population, using the reinitializepop function. This function takes popsize, n, and the bestpaths list variable as parameters. A population variable is initialized using the previous initializepop function but now, by taking ten percent of the popSize parameter, we randomly overwrite ten percent of the population with random high performing individuals then return this new modified population as Gen 0, to be utilized by the nested generations loop. Since we are maintaining a 90% level of randomness, we still are able to push the population in the correct direction for finding high fitness individuals while exploring other combinations. From here, once more, the outer WOC loop and nested generations loop continue to run until one of our stopping criteria is met.

After rigorous testing, it was decided that running the algorithm at 5 generations with 400 WOC loops which results in 2000 total generations produced the optimal output. Increasing the amount of generations but reducing the number of WOC loops produced a worse result as each new population was found to converge near the 5th generation mark; therefore, lowering the amount of WOC loops to increase the number of generations would result in less wisdom from the crowd and less exploration as the increase in generation would mostly be wasted on data that has already converged. Conversely, decreasing the amount of generations and increasing the amount of WOC loops had a similar effect given that each new population would not have enough time to progress to a point of convergence.

In regard to the GUI, this is where the pre-defined parameters of N queens, WOC loops, generations, population size, and mutation rate are defined by the user in entry boxes. This can be seen in figure 3, where we have the ability to click on run GA, and the processes mentioned earlier in this paper will be run. When the best combination is found, its fitness is displayed, and the combination is visualized on a chessboard. The user is allowed to zoom in, zoom out, and change between two of the best performing individuals if two were found. All of this code once again is defined within the main file.

Results

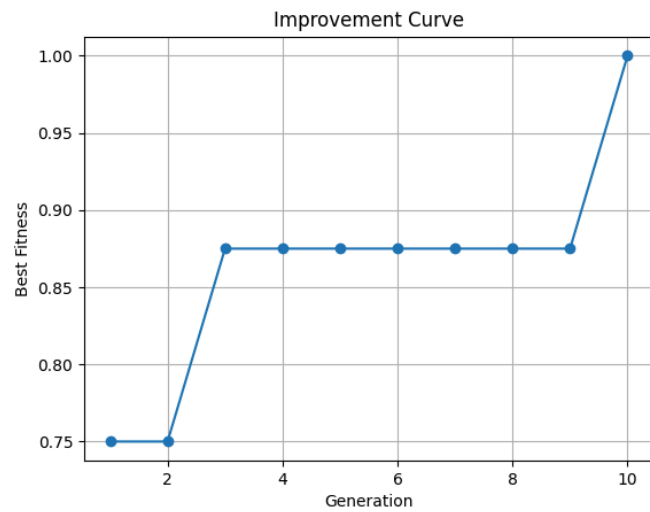
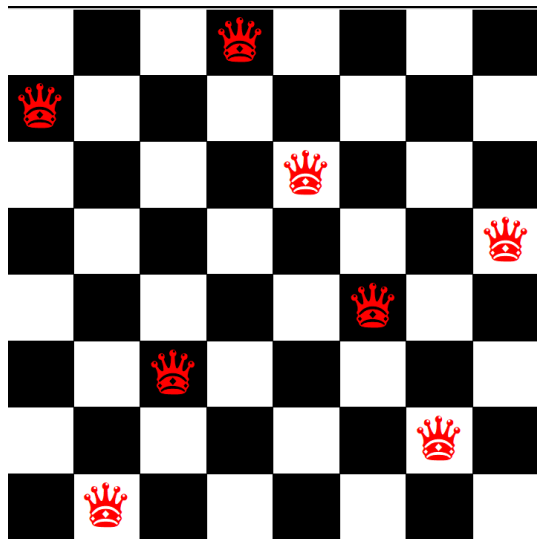


Fig 1.1: Example output for 8 queens with improvement curve

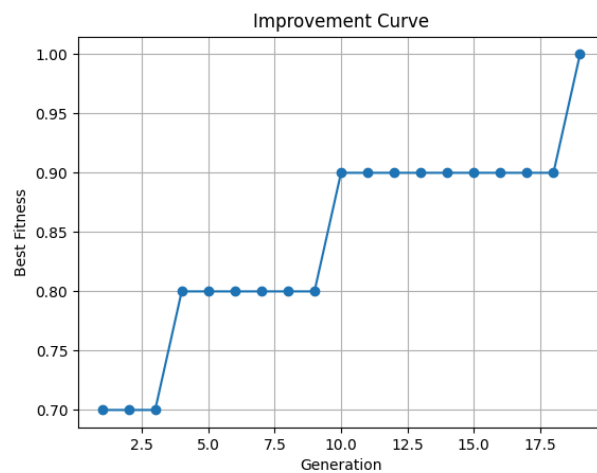
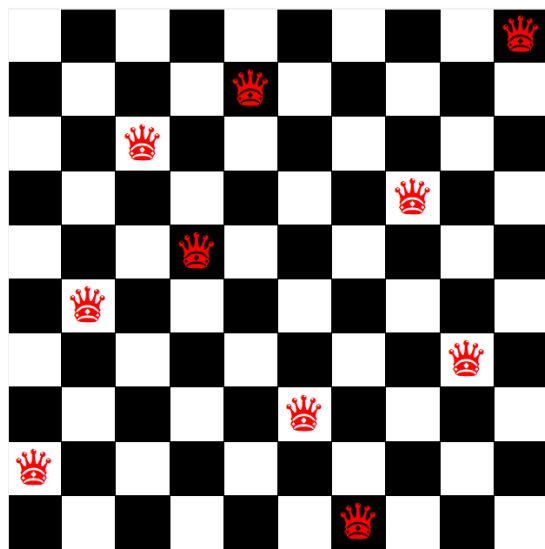


Fig 1.2: Example output for 10 queens with improvement curve

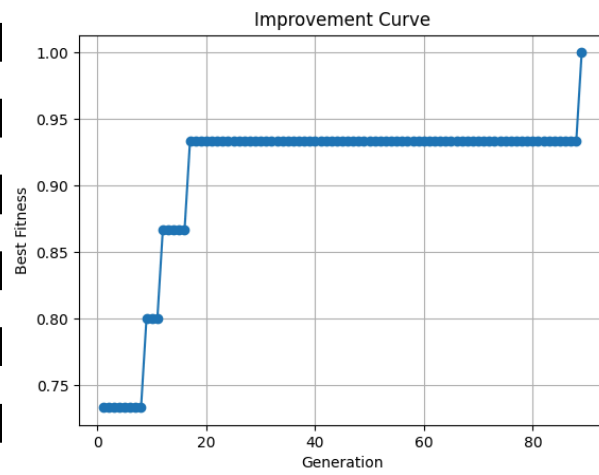
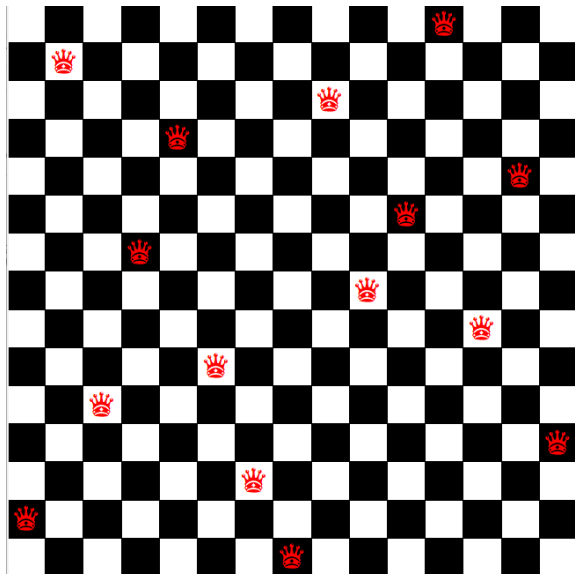


Fig 1.3: Example output for 15 queens with improvement curve

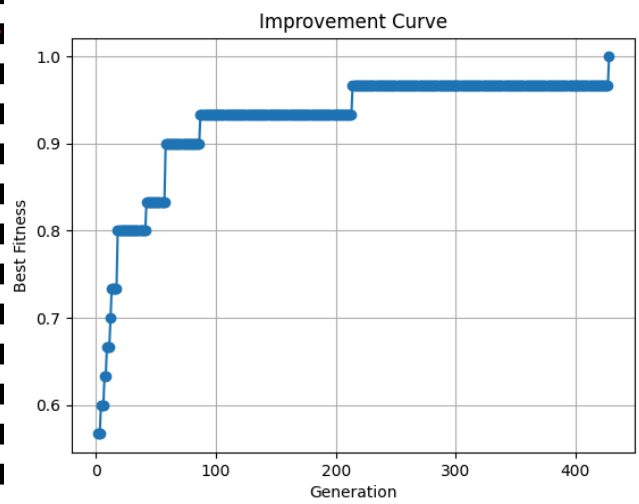
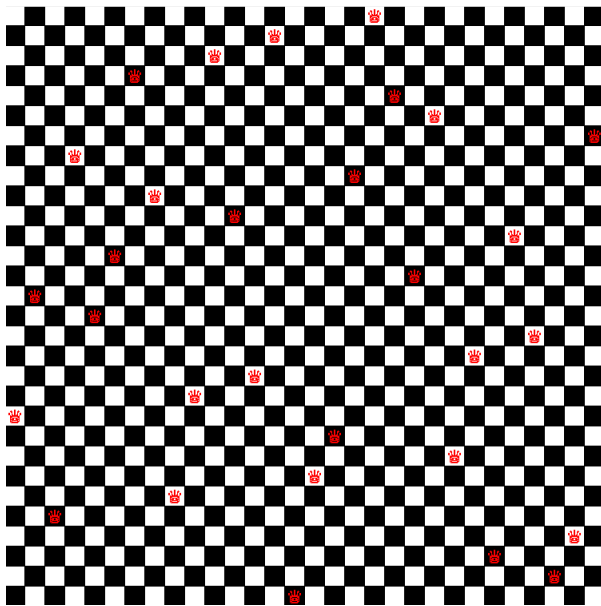


Fig 1.4: Example output for 30 queens with improvement curve

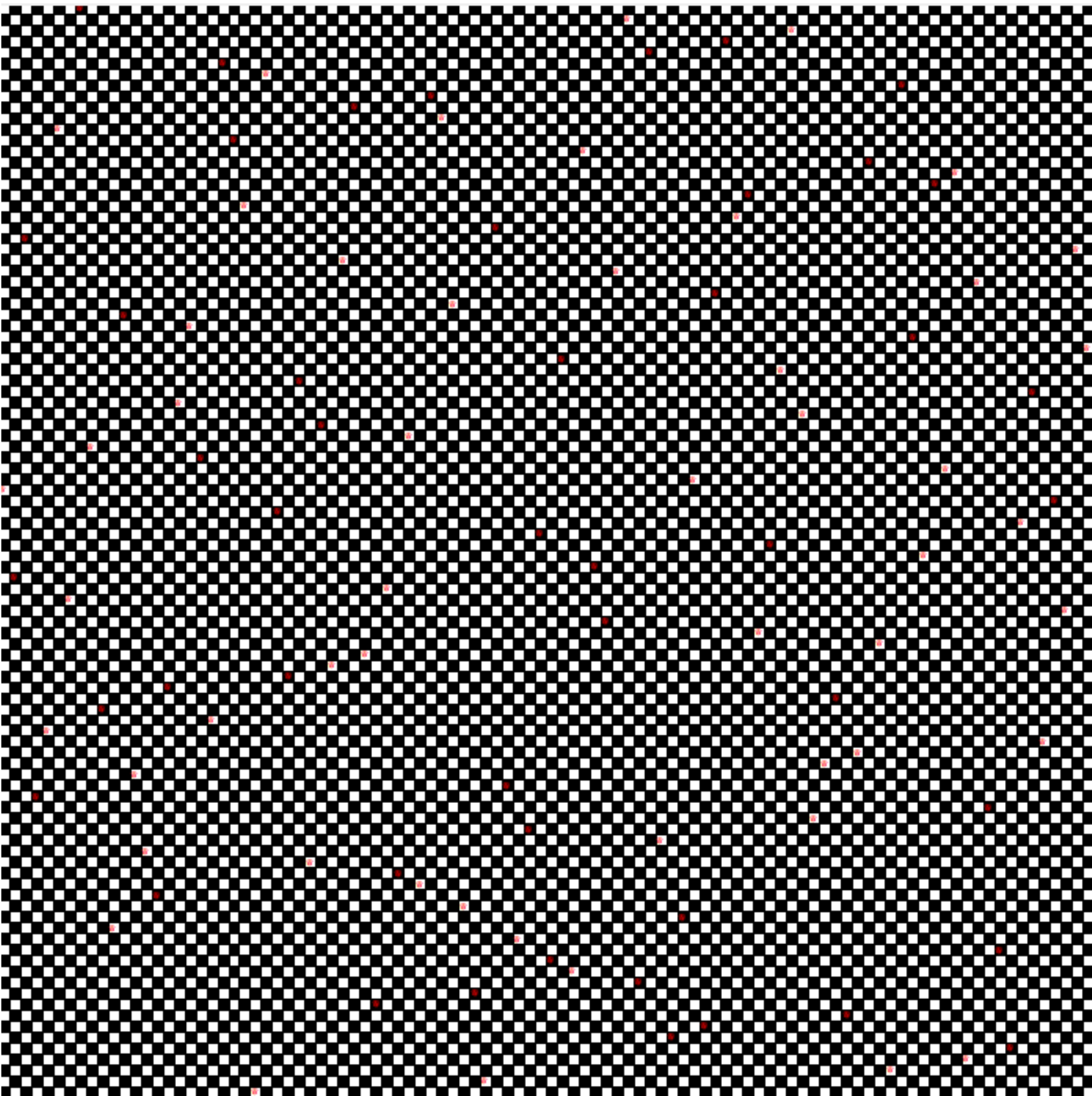


Fig 1.5: Example output for 100 queens

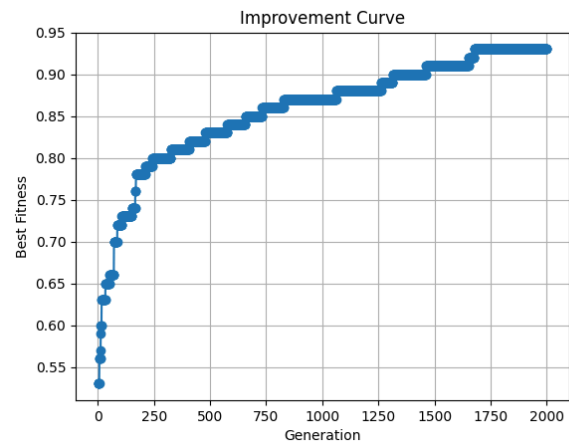


Fig 1.6: Improvement curve of output featured in figure 1.5

Fitness:	Standard 15	WoAC 15	Standard 25	WoAC 25	Standard 100	WoAC 100
	0.9333333333	1	0.84	1	0.65	0.9
	0.9333333333	0.9333333333	0.84	1	0.62	0.92
	1	0.9333333333	0.84	1	0.59	0.93
	0.9333333333	1	0.88	1	0.63	0.91
	0.9333333333	1	0.84	0.96	0.64	0.9
	0.9333333333	0.9333333333	0.84	0.96	0.64	0.93
	0.9333333333	1	0.84	1	0.62	0.88
	1	1	0.92	1	0.62	0.91
	0.9333333333	1	0.84	0.96	0.62	0.88
	0.9333333333	1	0.88	0.96	0.64	0.9
AVG	0.9466666667	0.98	0.856	0.984	0.627	0.906
MEDIAN	0.9333333333	1	0.84	1	0.625	0.905

Fig 2.1: Compilation of fitness data for standard genetic algorithm and hybrid WoAC genetic algorithm for multiple levels of N

Time:	Standard 15	WoAC 15	Standard 25	WoAC 25	Standard 100	WoAC 100
	1.389000893	0.2501187325	4.76794982	0.425999403	46.9815402	47.84262323
	1.500999689	0.1550004482	4.970208645	0.4985947609	47.01261997	49.15235066
	0.2990000248	0.07399940491	4.815333366	0.5681717396	47.57300115	49.84733891
	1.52600193	0.01500082016	4.568888187	0.4170005322	47.31081724	50.01046014
	1.448998213	1.740000725	4.797005177	4.485999346	46.7887857	48.50102401
	1.527979136	2.113999128	5.031111717	4.650146008	47.23531389	48.89795613
	1.411998034	0.009000778198	4.96202302	0.7359998226	47.15757012	49.57290912
	0.5089993477	0.08799934387	4.872472286	2.177999735	46.81028318	50.24411845
	1.428999662	0.02599978447	4.848542452	4.728136301	48.26928997	51.57561278
	1.424000502	2.125073433	4.720490217	4.696000338	46.49102259	49.16678381
AVG	1.246597743	0.6596192598	4.835402489	2.338404799	47.1630244	49.48111773
MEDIAN	1.426500082	0.121499896	4.831937909	1.456999779	47.08509505	49.36984646

Fig 2.2: Compilation of run time data in seconds for standard genetic algorithm and hybrid WoAC genetic algorithm for multiple levels of N

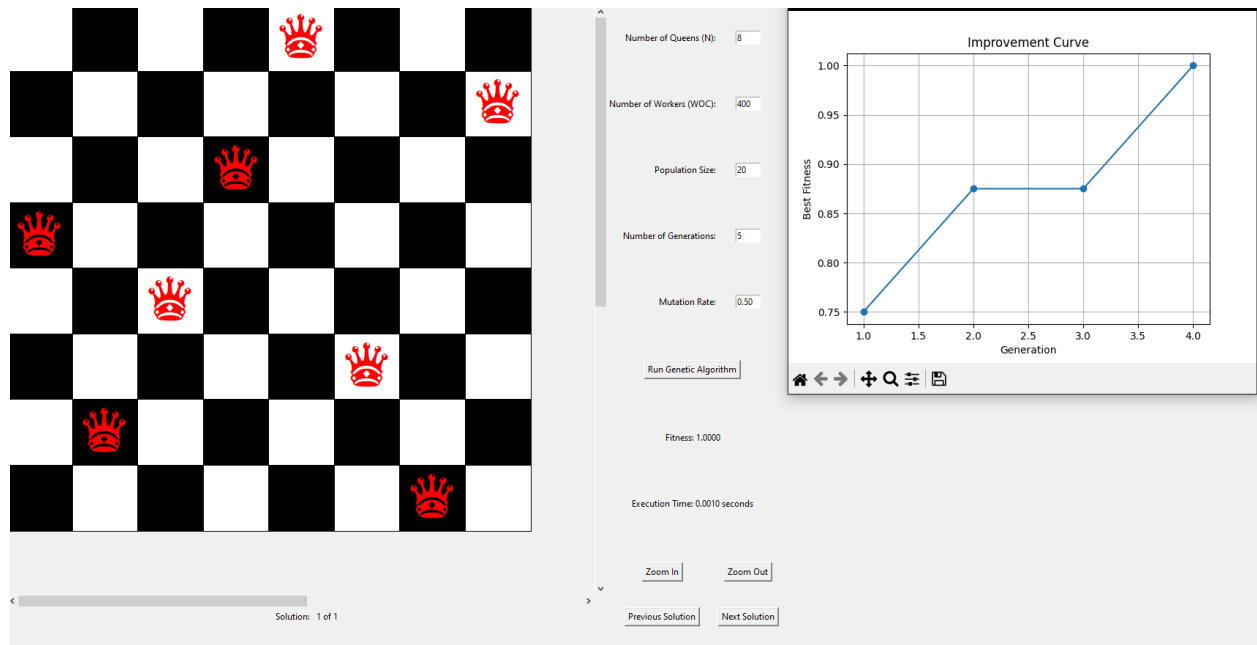


Fig. 3: GUI to visualize best found combination and improvement curve of fitnesses across generations.

Discussion

Test data was generated using a consistent input as seen in the results section. By differentiating input N's, running the algorithm multiple times, finding the average and median of each of these groups, and changing between the standard GA code with the hybrid wisdom of

crowds approach. This places every run under the same parameters every time, with only the implementation differentiating.

Observing the averages and medians produced in figures 2.1, and 2.2, more often than not, the hybrid approach performs better than standard GA. In figure 2.1, while average fitness improvement is marginal, when looking at median, hybrid actually tends to find a perfect solution far more often than the standard GA implementation; for 100 N's, however, for both average and median we find major improvement in solution fitness. In figure 2.2, the hybrid approach, aside from 100 N's, displays a fraction of standard GA's execution time for both the average and medians; for 100 N's however, this provides diminishing returns as there is a marginal increase in execution time for the hybrid approach relative to the standard, which could be explained by the algorithm actually finding a high performing solution as shown in figure 2.1, but taking a little longer to do so due to increased computational complexity. The reason for which the lower N's produce a lower execution time for the Wisdom of Crowds implementation is that the algorithm ceases upon finding a perfect solution, and the hybrid implementation had a higher propensity of finding perfect solutions than the standard GA. As N increases then we see a tradeoff in execution time for accuracy where the benefits far outweigh the costs.

In conclusion, The hybrid genetic algorithm with a wisdom of crowds approach ran as intended, with considerable improvement when comparing standard GA with the hybrid approach. On average, regardless of size n, fitness performs better with WOC implemented. When comparing execution times, however, there's diminishing returns as n grows. The WOC approach allows for gradual improvement and greater fitness collection.

For future experimentation and optimization there were a few considerations. A future implementation could include new functionality that could exclude already explored combinations overtime that have been documented as performing poorly. This would ensure that they are not part of reinitialized populations and re-explored. The list of paths being inserted when the population is reinitialized could be condensed to a list of lists, as to only hold the final generation run before reinitialization as by then, the combinations have converged on a high fitness combination. The algorithm could also be expanded to visualize multiple solutions